[INICIAL] TCArg 2026 Contest #2

A. Hate "A"

1 second, 256 megabytes

Bob has a string s consisting of lowercase English letters. He defines s' to be the string after removing all "a" characters from s (keeping all other characters in the same order). He then generates a new string t by concatenating s and s'. In other words, $t = s + s'$ (look at notes for an example).

You are given a string t. Your task is to find some s that Bob could have used to generate t. It can be shown that if an answer exists, it will be unique.

Input

The first line of input contains a string t ($1 \leq |t| \leq 10^5$) consisting of lowercase English letters.

Output

Print a string s that could have generated t. It can be shown if an answer exists, it is unique. If no string exists, print ":(" (without double quotes, there is no space between the characters).

input
```
aaaaa
```
output
```
aaaaa
```

input
```
aacaababc
```
output
```
:(
```

input
```
ababacacbbcc
```
output
```
ababacac
```

input
```
baba
```
output
```
:(
```

In the first example, we have $s =$ "aaaaa", and $s' =$ "".

In the second example, no such s can work that will generate the given t.

In the third example, we have $s =$ "ababacac", and $s' =$ "bbcc", and $t = s + s' =$ "ababacacbbcc".

B. Nauuo and Cards

1.5 seconds, 256 megabytes

Nauuo is a girl who loves playing cards.

One day she was playing cards but found that the cards were mixed with some empty ones.

There are n cards numbered from 1 to n, and they were mixed with another n empty cards. She piled up the $2n$ cards and drew n of them. The n cards in Nauuo's hands are given. The remaining n cards in the pile are also given in the order from top to bottom.

In one operation she can choose a card in her hands and play it — put it at the bottom of the pile, then draw the top card from the pile.

Nauuo wants to make the n numbered cards piled up in increasing order (the i-th card in the pile from top to bottom is the card i) as quickly as possible. Can you tell her the minimum number of operations?

Input

The first line contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of numbered cards.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($0 \leq a_i \leq n$) — the initial cards in Nauuo's hands. 0 represents an empty card.

The third line contains n integers $b_1, b_2, \ldots, b_n$ ($0 \leq b_i \leq n$) — the initial cards in the pile, given in order from top to bottom. 0 represents an empty card.

It is guaranteed that each number from 1 to n appears exactly once, either in $a_{1..n}$ or $b_{1..n}$.

Output

The output contains a single integer — the minimum number of operations to make the n numbered cards piled up in increasing order.

input
```
3
0 2 0
3 0 1
```
output
```
2
```

input
```
3
0 2 0
1 0 3
```
output
```
4
```

input
```
11
0 0 0 5 0 0 0 4 0 0 11
9 2 6 0 8 1 7 0 3 0 10
```
output
```
18
```

Example 1

We can play the card 2 and draw the card 3 in the first operation. After that, we have $[0, 3, 0]$ in hands and the cards in the pile are $[0, 1, 2]$ from top to bottom.

Then, we play the card 3 in the second operation. The cards in the pile are $[1, 2, 3]$, in which the cards are piled up in increasing order.

Example 2

Play an empty card and draw the card 1, then play 1, 2, 3 in order.

C. Andryusha and Socks

2 seconds, 256 megabytes

Andryusha is an orderly boy and likes to keep things in their place.

Today he faced a problem to put his socks in the wardrobe. He has n distinct pairs of socks which are initially in a bag. The pairs are numbered from 1 to n. Andryusha wants to put paired socks together and put them in the wardrobe. He takes the socks one by one from the bag, and for each sock he looks whether the pair of this sock has been already took out of the bag, or not. If not (that means the pair of this sock is still in the bag), he puts the current socks on the table in front of him. Otherwise, he puts both socks from the pair to the wardrobe.

Andryusha remembers the order in which he took the socks from the bag. Can you tell him what is the maximum number of socks that were on the table at the same time?

Input

The first line contains the single integer n ($1 \le n \le 10^5$) — the number of sock pairs.

The second line contains $2n$ integers $x_1, x_2, ..., x_{2n}$ ($1 \le x_i \le n$), which describe the order in which Andryusha took the socks from the bag. More precisely, x_i means that the i-th sock Andryusha took out was from pair x_i.

It is guaranteed that Andryusha took exactly two socks of each pair.

Output

Print single integer — the maximum number of socks that were on the table at the same time.

input
```
1
1 1
```
output
```
1
```

input
```
3
2 1 1 3 2 3
```
output
```
2
```

In the first example Andryusha took a sock from the first pair and put it on the table. Then he took the next sock which is from the first pair as well, so he immediately puts both socks to the wardrobe. Thus, at most one sock was on the table at the same time.

In the second example Andryusha behaved as follows:

- Initially the table was empty, he took out a sock from pair 2 and put it on the table.
- Sock (2) was on the table. Andryusha took out a sock from pair 1 and put it on the table.
- Socks $(1, 2)$ were on the table. Andryusha took out a sock from pair 1, and put this pair into the wardrobe.
- Sock (2) was on the table. Andryusha took out a sock from pair 3 and put it on the table.
- Socks $(2, 3)$ were on the table. Andryusha took out a sock from pair 2, and put this pair into the wardrobe.
- Sock (3) was on the table. Andryusha took out a sock from pair 3 and put this pair into the wardrobe.

Thus, at most two socks were on the table at the same time.

D. Andryusha and Colored Balloons

2 seconds, 256 megabytes

Andryusha goes through a park each day. The squares and paths between them look boring to Andryusha, so he decided to decorate them.

The park consists of n squares connected with $(n - 1)$ bidirectional paths in such a way that any square is reachable from any other using these paths. Andryusha decided to hang a colored balloon at each of the squares. The baloons' colors are described by positive integers, starting from 1. In order to make the park varicolored, Andryusha wants to choose the colors in a special way. More precisely, he wants to use such colors that if a, b and c are distinct squares that a and b have a direct path between them, and b and c have a direct path between them, then balloon colors on these three squares are distinct.

Andryusha wants to use as little different colors as possible. Help him to choose the colors!

Input

The first line contains single integer n ($3 \le n \le 2 \cdot 10^5$) — the number of squares in the park.

Each of the next $(n - 1)$ lines contains two integers x and y ($1 \le x, y \le n$) — the indices of two squares directly connected by a path.

It is guaranteed that any square is reachable from any other using the paths.

Output

In the first line print single integer k — the minimum number of colors Andryusha has to use.

In the second line print n integers, the i-th of them should be equal to the balloon color on the i-th square. Each of these numbers should be within range from 1 to k.

input
```
3
2 3
1 3
```
output
```
3
1 3 2
```

input
```
5
2 3
5 3
4 3
1 3
```
output
```
5
1 3 2 5 4
```

input
```
5
2 1
3 2
4 3
5 4
```
output
```
3
1 2 3 1 2
```

In the first sample the park consists of three squares: $1 \to 3 \to 2$. Thus, the balloon colors have to be distinct.

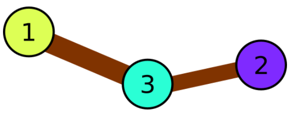

Illustration for the first sample.

In the second example there are following triples of consequently connected squares:

- $1 \to 3 \to 2$
- $1 \to 3 \to 4$
- $1 \to 3 \to 5$
- $2 \to 3 \to 4$
- $2 \to 3 \to 5$
- $4 \to 3 \to 5$

We can see that each pair of squares is encountered in some triple, so all colors have to be distinct.

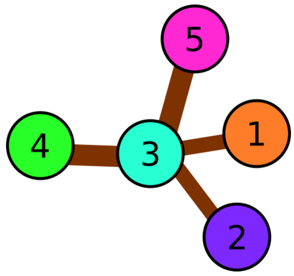

Illustration for the second sample.

In the third example there are following triples:

- $1 \to 2 \to 3$
- $2 \to 3 \to 4$
- $3 \to 4 \to 5$

We can see that one or two colors is not enough, but there is an answer that uses three colors only.

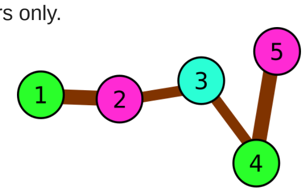

Illustration for the third sample.

E. Minimax Problem

5 seconds, 512 megabytes

You are given n arrays a_1, a_2, ..., a_n; each array consists of exactly m integers. We denote the y-th element of the x-th array as $a_{x,y}$.

You have to choose two arrays a_i and a_j ($1 \le i, j \le n$, it is possible that $i = j$). After that, you will obtain a new array b consisting of m integers, such that for every $k \in [1, m]$ $b_k = \max(a_{i,k}, a_{j,k})$.

Your goal is to choose i and j so that the value of $\min\limits_{k=1}^{m} b_k$ is maximum possible.

Input

The first line contains two integers n and m ($1 \le n \le 3 \cdot 10^5$, $1 \le m \le 8$) — the number of arrays and the number of elements in each array, respectively.

Then n lines follow, the x-th line contains the array a_x represented by m integers $a_{x,1}$, $a_{x,2}$, ..., $a_{x,m}$ ($0 \le a_{x,y} \le 10^9$).

Output

Print two integers i and j ($1 \le i, j \le n$, **it is possible that** $i = j$) — the indices of the two arrays you have to choose so that the value of $\min\limits_{k=1}^{m} b_k$ is maximum possible. If there are multiple answers, print any of them.

input

```
6 5
5 0 3 1 2
1 8 9 1 3
1 2 3 4 5
9 1 0 3 7
2 3 0 6 3
6 4 1 7 0
```

output

```
1 5
```

F. Shuffle Hashing

2 seconds, 256 megabytes

Polycarp has built his own web service. Being a modern web service it includes login feature. And that always implies password security problems.

Polycarp decided to store the hash of the password, generated by the following algorithm:

1. take the password p, consisting of lowercase Latin letters, and shuffle the letters randomly in it to obtain p' (p' can still be equal to p);
2. generate two random strings, consisting of lowercase Latin letters, s_1 and s_2 (any of these strings can be empty);
3. the resulting hash $h = s_1 + p' + s_2$, where addition is string concatenation.

For example, let the password $p =$ "abacaba". Then p' can be equal to "aabcaab". Random strings $s1 =$ "zyx" and $s2 =$ "kjh". Then $h =$ "zyxaabcaabkjh".

Note that no letters could be deleted or added to p to obtain p', only the order could be changed.

Now Polycarp asks you to help him to implement the password check module. Given the password p and the hash h, check that h can be the hash for the password p.

Your program should answer t independent test cases.

Input

The first line contains one integer t ($1 \le t \le 100$) — the number of test cases.

The first line of each test case contains a non-empty string p, consisting of lowercase Latin letters. The length of p does not exceed 100.

The second line of each test case contains a non-empty string h, consisting of lowercase Latin letters. The length of h does not exceed 100.

Output

For each test case print the answer to it — "YES" if the given hash h could be obtained from the given password p or "NO" otherwise.

input

```
5
abacaba
zyxaabcaabkjh
onetwothree
threetwoone
one
zzonneyy
one
none
twenty
ten
```

output

```
YES
YES
NO
YES
NO
```

The first test case is explained in the statement.

In the second test case both s_1 and s_2 are empty and $p' =$ "threetwoone" is p shuffled.

In the third test case the hash could not be obtained from the password.

In the fourth test case $s_1 =$ "n", s_2 is empty and $p' =$ "one" is p shuffled (even thought it stayed the same).

In the fifth test case the hash could not be obtained from the password.

G. Messenger Simulator

3 seconds, 256 megabytes

Polycarp is a frequent user of the very popular messenger. He's chatting with his friends all the time. He has n friends, numbered from 1 to n.

Recall that a permutation of size n is an array of size n such that each integer from 1 to n occurs exactly once in this array.

So his recent chat list can be represented with a permutation p of size n. p_1 is the most recent friend Polycarp talked to, p_2 is the second most recent and so on.

Initially, Polycarp's recent chat list p looks like $1, 2, \dots, n$ (in other words, it is an identity permutation).

After that he receives m messages, the j-th message comes from the friend a_j. And that causes friend a_j to move to the first position in a permutation, shifting everyone between the first position and the current position of a_j by 1. Note that if the friend a_j is in the first position already then nothing happens.

For example, let the recent chat list be $p = [4, 1, 5, 3, 2]$:

- if he gets messaged by friend 3, then p becomes $[3, 4, 1, 5, 2]$;
- if he gets messaged by friend 4, then p doesn't change $[4, 1, 5, 3, 2]$;
- if he gets messaged by friend 2, then p becomes $[2, 4, 1, 5, 3]$.

For each friend consider all position he has been at in the beginning and after receiving each message. Polycarp wants to know what were the minimum and the maximum positions.

Input

The first line contains two integers n and m ($1 \le n, m \le 3 \cdot 10^5$) — the number of Polycarp's friends and the number of received messages, respectively.

The second line contains m integers $a_1, a_2, \dots, a_m$ ($1 \le a_i \le n$) — the descriptions of the received messages.

Output

Print n pairs of integers. For each friend output the minimum and the maximum positions he has been in the beginning and after receiving each message.

input

```
5 4
3 5 1 4
```

output

```
1 3
2 5
1 4
1 5
1 5
```

input

```
4 3
1 2 4
```

output

```
1 3
1 2
3 4
1 4
```

In the first example, Polycarp's recent chat list looks like this:

- $[1, 2, 3, 4, 5]$
- $[3, 1, 2, 4, 5]$
- $[5, 3, 1, 2, 4]$
- $[1, 5, 3, 2, 4]$
- $[4, 1, 5, 3, 2]$

So, for example, the positions of the friend 2 are $2, 3, 4, 4, 5$, respectively. Out of these 2 is the minimum one and 5 is the maximum one. Thus, the answer for the friend 2 is a pair $(2, 5)$.

In the second example, Polycarp's recent chat list looks like this:

- $[1, 2, 3, 4]$
- $[1, 2, 3, 4]$
- $[2, 1, 3, 4]$
- $[4, 2, 1, 3]$

H. Division and Union

2 seconds, 256 megabytes

There are n segments $[l_i, r_i]$ for $1 \le i \le n$. You should divide all segments into two *non-empty* groups in such way that there is no pair of segments from different groups which have at least one common point, or say that it's impossible to do it. Each segment should belong to exactly one group.

To optimize testing process you will be given multitest.

Input

The first line contains one integer T ($1 \le T \le 50000$) — the number of queries. Each query contains description of the set of segments. Queries are independent.

First line of each query contains single integer n ($2 \le n \le 10^5$) — number of segments. It is guaranteed that $\sum n$ over all queries does not exceed 10^5.

The next n lines contains two integers l_i, r_i per line ($1 \le l_i \le r_i \le 2 \cdot 10^5$) — the i-th segment.

Output

For each query print n integers $t_1, t_2, \dots, t_n$ ($t_i \in \{1, 2\}$) — for each segment (in the same order as in the input) t_i equals 1 if the i-th segment will belongs to the first group and 2 otherwise.

If there are multiple answers, you can print any of them. If there is no answer, print -1.

input

```
3
2
5 5
2 3
3
3 5
2 3
2 3
3
3 3
4 4
5 5
```

output

```
2 1
-1
1 1 2
```

In the first query the first and the second segments should be in different groups, but exact numbers don't matter.

In the second query the third segment intersects with the first and the second segments, so they should be in the same group, but then the other group becomes empty, so answer is -1.

In the third query we can distribute segments in any way that makes groups non-empty, so any answer of 6 possible is correct.

I. Long Number

2 seconds, 256 megabytes

You are given a long decimal number a consisting of n digits from 1 to 9. You also have a function f that maps every digit from 1 to 9 to some (possibly the same) digit from 1 to 9.

You can perform the following operation **no more than once**: choose a non-empty **contiguous subsegment** of digits in a, and replace each digit x from this segment with $f(x)$. For example, if $a = 1337$, $f(1) = 1$, $f(3) = 5$, $f(7) = 3$, and you choose the segment consisting of three rightmost digits, you get 1553 as the result.

What is the maximum possible number you can obtain applying this operation no more than once?

Input

The first line contains one integer n ($1 \le n \le 2 \cdot 10^5$) — the number of digits in a.

The second line contains a string of n characters, denoting the number a. Each character is a decimal digit from 1 to 9.

The third line contains exactly 9 integers $f(1)$, $f(2)$, ..., $f(9)$ ($1 \le f(i) \le 9$).

Output

Print the maximum number you can get after applying the operation described in the statement no more than once.

input
```
4
1337
1 2 5 4 6 6 3 1 9
```
output
```
1557
```

input
```
5
11111
9 8 7 6 5 4 3 2 1
```
output
```
99999
```

input
```
2
33
1 1 1 1 1 1 1 1 1
```
output
```
33
```

J. Jongmah

3 seconds, 256 megabytes

You are playing a game of Jongmah. You don't need to know the rules to solve this problem. You have n tiles in your hand. Each tile has an integer between 1 and m written on it.

To win the game, you will need to form some number of *triples*. Each triple consists of three tiles, such that the numbers written on the tiles are either all the same or consecutive. For example, $7, 7, 7$ is a valid triple, and so is $12, 13, 14$, but $2, 2, 3$ or $2, 4, 6$ are not. You can only use the tiles in your hand to form triples. Each tile can be used in at most one triple.

To determine how close you are to the win, you want to know the maximum number of triples you can form from the tiles in your hand.

Input

The first line contains two integers integer n and m ($1 \le n, m \le 10^6$) — the number of tiles in your hand and the number of tiles types.

The second line contains integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le m$), where a_i denotes the number written on the i-th tile.

Output

Print one integer: the maximum number of triples you can form.

input
```
10 6
2 3 3 3 4 4 4 5 5 6
```
output
```
3
```

input
```
12 6
1 5 3 3 3 4 3 5 3 2 3 3
```
output
```
3
```

input
```
13 5
1 1 5 1 2 3 3 2 4 2 3 4 5
```
output
```
4
```

In the first example, we have tiles $2, 3, 3, 3, 4, 4, 4, 5, 5, 6$. We can form three triples in the following way: $2, 3, 4$; $3, 4, 5$; $4, 5, 6$. Since there are only 10 tiles, there is no way we could form 4 triples, so the answer is 3.

In the second example, we have tiles 1, 2, 3 (7 times), 4, 5 (2 times). We can form 3 triples as follows: $1, 2, 3$; $3, 3, 3$; $3, 4, 5$. One can show that forming 4 triples is not possible.

K. Game with Telephone Numbers

1 second, 256 megabytes

A telephone number is a sequence of **exactly** 11 digits such that its first digit is 8.

Vasya and Petya are playing a game. Initially they have a string s of length n (n is odd) consisting of digits. Vasya makes the first move, then players alternate turns. In one move the player **must** choose a character and erase it from the current string. For example, if the current string `1121`, after the player's move it may be `112`, `111` or `121`. The game ends when the length of string s becomes `11`. If the resulting string is a telephone number, Vasya wins, otherwise Petya wins.

You have to determine if Vasya has a winning strategy (that is, if Vasya can win the game no matter which characters Petya chooses during his moves).

Input

The first line contains one integer n ($13 \le n < 10^5$, n is odd) — the length of string s.

The second line contains the string s ($|s| = n$) consisting only of decimal digits.

Output

If Vasya has a strategy that guarantees him victory, print YES.

Otherwise print `NO`.

input
```
13
8380011223344
```
output
```
YES
```

input
```
15
807345619350641
```
output
```
NO
```

In the first example Vasya needs to erase the second character. Then Petya cannot erase a character from the remaining string `880011223344` so that it does not become a telephone number.

In the second example after Vasya's turn Petya can erase one character character 8. The resulting string can't be a telephone number, because there is no digit 8 at all.

L. Nauuo and Votes

1 second, 256 megabytes

Nauuo is a girl who loves writing comments.

One day, she posted a comment on Codeforces, wondering whether she would get upvotes or downvotes.

It's known that there were x persons who would upvote, y persons who would downvote, and there were also another z persons who would vote, but you don't know whether they would upvote or downvote. Note that each of the $x + y + z$ people would vote exactly one time.

There are three different results: if there are more people upvote than downvote, the result will be "+"; if there are more people downvote than upvote, the result will be "-"; otherwise the result will be "0".

Because of the z unknown persons, the result may be uncertain (i.e. there are more than one possible results). More formally, the result is uncertain if and only if there exist two different situations of how the z persons vote, that the results are different in the two situations.

Tell Nauuo the result or report that the result is uncertain.

Input

The only line contains three integers x, y, z $(0 \le x, y, z \le 100)$, corresponding to the number of persons who would upvote, downvote or unknown.

Output

If there is **only one** possible result, print the result : "+", "-" or "0".

Otherwise, print "?" to report that the result is uncertain.

input
```
3 7 0
```
output
```
-
```

input
```
2 0 1
```
output
```
+
```

input
```
1 1 0
```
output
```
0
```

input
```
0 0 1
```
output
```
?
```

In the first example, Nauuo would definitely get three upvotes and seven downvotes, so the only possible result is "-".

In the second example, no matter the person unknown downvotes or upvotes, Nauuo would get more upvotes than downvotes. So the only possible result is "+".

In the third example, Nauuo would definitely get one upvote and one downvote, so the only possible result is "0".

In the fourth example, if the only one person upvoted, the result would be "+", otherwise, the result would be "-". There are two possible results, so the result is uncertain.